

Submission Worksheet

IT202-450-M2026 / it202-milestone-1-2026

Submission Data

Course	IT202-450-M2026
Assignment	IT202 Milestone 1 - 2026
Student	Michos C. (mrc82)
Status	submitted
Worksheet Progress	100%
Potential Grade	NaN/NaN (NaN%)
Updated	7/18/2026, 6:31:55 PM

Grading Link

<https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/it202-milestone-1-2026/grading/mrc82>

View Link

<https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/it202-milestone-1-2026/mrc82>

Instructions

Verify that your lesson-built Milestone 1 account system is complete and functional, then collect focused evidence for the account, access, role, profile, and styling requirements. Most of the coding should already be done from the lessons; this worksheet focuses on confirming the required behavior, polishing the project CSS, and explaining your understanding.

Assignment Setup

- Required branch: `Milestone1`
- Required starting branch: `qa`
- Required project folder: `public_html/project/`
- Required deployed evidence target: working `qa` site before production promotion
- Recommended order:
 - Read the whole worksheet.
 - Review the Milestone 1 assignment requirements.
 - Check out `qa`, pull the latest version, and create the milestone branch.
 - Ensure the related lessons are complete in your project.
 - Test each feature and requirement.
 - Apply your custom CSS to replace the temporary lesson example CSS.
 - Merge the milestone branch into `qa`, test deployed behavior, then collect worksheet evidence.

Git Workflow Checklist

1. Check out `qa`.
2. Pull the latest `qa` branch with `git pull origin qa`.
3. Create the milestone branch with `git checkout -b Milestone1`.
4. Commit focused verification, fixes, and styling updates in small pieces. If there's nothing to commit, wait until you add the PDF to your repository.
5. Push `Milestone1`.
6. Open a pull request from `Milestone1` into `qa` and keep it open while you work.

- add the PDF to your repository.
5. Push `Milestone1`.
6. Open a pull request from `Milestone1` into `qa` and keep it open while you work.
7. After all Milestone 1 requirements pass locally, merge the pull request into `qa`.
8. Gather evidence from the deployed `qa` site and fill out the worksheet.
9. Export the worksheet PDF from the Learn Courses Platform.
10. Save the PDF in the repository under `docs/milestone01/`.
11. Add, commit, and push the PDF to the `Milestone1` branch.
 - `git add .`
 - `git commit -m "Add milestone 1 worksheet PDF"`
 - `git push origin Milestone1`
12. Upload the same PDF to Canvas.
13. Create and merge a pull request from `qa` into `prod` when production evidence is required.
14. Switch back to `qa`.
15. Pull the latest `qa` branch.

Shared Requirements

Apply these requirements to every Milestone 1 feature:

- Add a comment with your UCID, date, and a brief summary of the code block or feature area.
- Follow the shared helper, validation, flash message, session, and role-check patterns from the lessons.
- Replace the temporary lesson CSS with your own project styling.
- Use user-friendly flash messages for success and failure states.
- Log technical errors with `error_log()` instead of showing raw technical output to users.
- Use HTML validation attributes, JavaScript validation through the page's `validate()` function, and PHP validation before database writes.
- Keep safe form values sticky after validation errors.
- Clear password fields after validation errors.
- Test pages through the supported deployed `qa` URLs before collecting browser evidence.
- Use your own code, database behavior, branch, pull request, deployed URLs, and explanation evidence.

Evidence Format

Most sections use the same evidence pattern:

- Images:
 - Show relevant code and matching browser output.
 - Make sure UCID/date comments are visible in code evidence.
 - Make sure the deployed `qa` URL is visible in browser output evidence.
 - One combined screenshot is acceptable when the code and output are both readable.
 - Use multiple screenshots when one image would be hard to read.
- URLs:
 - Provide direct GitHub file links from the milestone branch.
 - Provide anticipated production URLs for deployed PHP pages.
 - Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
 - PHP page URLs must end in `.php`; zero credit for that URL entry if it does not.
- Response:
 - Explain the logic in your own words.
 - Describe the steps or decisions your PHP, SQL, or JavaScript makes.
 - Do not restate the prompt or list the requirements.

Submission Check

- Confirm the Milestone 1 branch has been pushed.
- Confirm the pull request into `qa` exists

Submission Check

- Confirm the Milestone 1 branch has been pushed.
- Confirm the pull request into **qa** exists.
- Confirm meaningful Milestone 1 commits are visible.
- Confirm the deployed **qa** version was reviewed before promotion.
- Confirm registration, login, logout, protected access, role access, profile edit, and password change were tested.
- Confirm the temporary lesson CSS has been replaced with your own styling.
- Confirm anticipated production URLs use the same path as the tested **qa** URLs with **-qa** changed to **-prod**.
- Confirm the worksheet PDF is committed under **docs/milestone01/** on the milestone branch.
- Upload the same worksheet PDF to Canvas.

Submission Progress

100%

Section #1: (1 pt.) Account Database

100%

- Goal: show that the account system has the expected database structure.
- Expected database behavior:
 - **Users** stores username, email, password hash, created, and modified values.
 - Username and email are unique.
 - **Roles** stores available roles.
 - **UserRoles** connects users and roles.

Task #1 (1 pt.) – Account Table Evidence

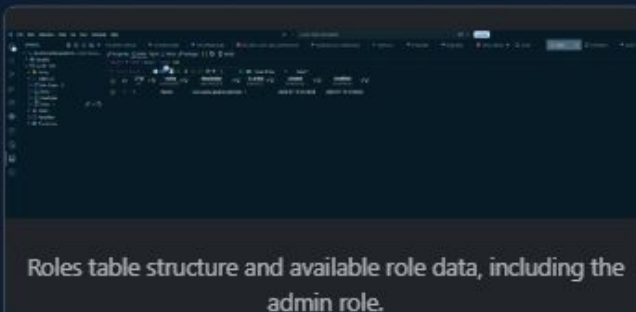
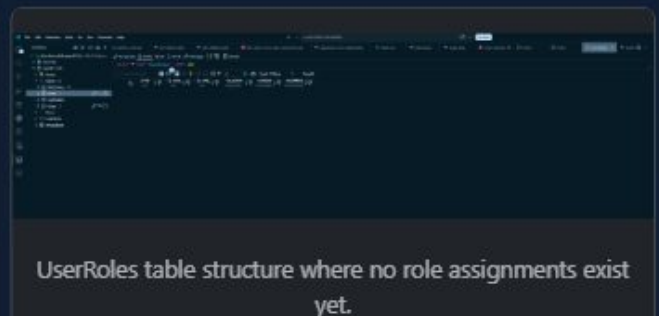
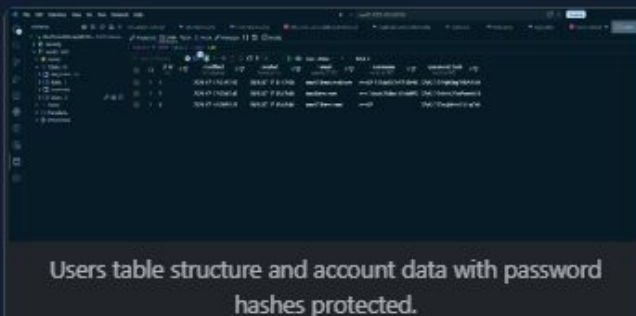
100%

Combo #1

Part 1: Images 100%

Details

- Show the **Users** table from the VS Code database extension so the structure and some data are visible.
- Show the **Roles** table from the VS Code database extension so the structure and some data are visible.
- Show the **UserRoles** table from the VS Code database extension so the structure and some data are visible.



Part 2: URLs 100%

Part 2: URLs 100%

Details

- Paste the direct GitHub file link for `sql/001_create_users.sql`.
- Paste the direct GitHub file link for `sql/003_create_table_roles.sql`.
- Paste the direct GitHub file link for `sql/004_create_table_userroles.sql`.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/prod/sql/001_create_users.sql 🍏

URL #2

https://github.com/micopher/mrc82-IT202-450-M2026/blob/prod/sql/003_create_table_roles.sql 🍏

URL #3

https://github.com/micopher/mrc82-IT202-450-M2026/blob/prod/sql/004_create_table_userroles.sql 🍏

Part 3: Response 100%

Details

- Explain how the three tables work together.
- Explain where username and email uniqueness are enforced.

Response

The Users table stores each account's username, email, password hash, and timestamps. The Roles table stores the available roles, such as the admin role. The UserRoles table connects users to roles using the corresponding user and role IDs, allowing roles to be assigned to individual accounts. Username and email uniqueness are enforced by UNIQUE constraints in the Users table, preventing duplicate values.

Supports **bold**, *italic*, [links](url), ``code``, etc.

408 chars

Section #2: (1 pt.) Shared Setup And Helpers

100%

- Goal: show that the project loads and uses reusable account helpers from a shared setup file.
- Expected helper behavior:
 - Shared app setup loads the database, session, validation helpers, user helpers, role helpers, flash helpers, and other project utilities.
 - User helper functions safely check login state before reading user session values.
 - Role helper functions can check the current user's roles.

Task #1 (1 pt.) – Shared Helper Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Show `lib/app.php` with the shared helper imports visible.
- The screenshot should show references to the database, session, validation, flash, user, role, render, duplicate-user-detail, and URL helper files.





Shared app setup loading the session and all reusable project helper files.

Part 2: URLs 100%

Details

- Paste the direct GitHub file link for `lib/app.php`.

URL #1

<https://github.com/micopher/mrc82-IT202-450-M2026/blob/prod/lib/app.php> 🍎

Part 3: Response 100%

Details

- Explain how `lib/app.php` loads the shared helper files.
- Explain why the specific load order matters.

Response

`lib/app.php` starts the session and uses `require_once` to load the database, rendering, URL, validation, user, flash-message, duplicate-user-detail, and role helpers. This makes the shared functions available to every project page that loads `app.php`.

Supports **bold**, *italic*, [links\(url\)](#), ``code``, etc.

248 chars

Section #3: (2 pts.) Registration

100%

- Goal: show that users can register with username, email, password, and confirm password.
- Expected behavior:
 - HTML, JavaScript, and PHP validation are all present.
 - Duplicate username and duplicate email cases show specific, user-friendly feedback.
 - Passwords are stored as hashes.
 - Safe form values remain sticky after validation errors.

Task #1 (2 pts.) – Registration Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Show the registration form code, including username, email, password, confirm password, and the escaped username pattern.
- Show the JavaScript `validate()` logic for registration.
- Show the PHP validation and insert logic, including password hashing and duplicate handling.
- Show deployed qa browser evidence for successful registration, duplicate username, duplicate email, and one

- Show the JavaScript `validate()` logic for registration.
- Show the PHP validation and insert logic, including password hashing and duplicate handling.
- Show deployed `qa` browser evidence for successful registration, duplicate username, duplicate email, and one validation failure.



Registration fields with required validation attributes and escaped sticky values.



JavaScript registration validation for username, email, password, and password confirmation.



PHP validation, bcrypt password hashing, prepared insertion, and duplicate account handling.



Successful account registration on the deployed QA site with confirmation feedback.



Duplicate username registration attempt showing specific feedback on the deployed QA site.



Duplicate email registration attempt showing specific feedback on the deployed QA site.



JavaScript validation rejecting invalid registration values on the deployed QA site.

Part 2: URLs 100%

Details

- Paste the direct GitHub file link for `public_html/project/register.php`.
- Paste the anticipated production URL for the registration page.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/prod/public_html/project/register.php 🟢

URL #2

<https://mrc82-it202-450-m2026-prod.onrender.com/project/register.php> 🟢

Part 3: Response 100%

Part 3: Response 100%

Details

- Explain how registration validates input across HTML, JavaScript, and PHP.
- Explain how duplicate username and email feedback is shown.
- Explain how the password is stored safely.

Response

The registration form first checks input with HTML attributes such as required, minlength, maxlength, pattern, and the email input type. JavaScript then runs the shared validation functions before submission, while PHP validates the values again before inserting them into the database. If the username or email already exists, the duplicate handler displays a specific user-friendly message. The password is hashed with `password_hash()` using `bcrypt`, so only the hash is stored instead of the actual password.

Supports **bold**, *italic*, [links](url), `code`, etc.

509 chars

Section #4: (2 pts.) Login And Logout

100%

- Goal: show that users can log in by username or email and fully log out.
- Expected login behavior:
 - Login uses one identifier field.
 - The server-side lookup safely checks username or email.
 - Passwords are verified with `password_verify()`.
 - The session stores the user's safe details and roles.
- Expected logout behavior:
 - Logout destroys the session.
 - Logout redirects with a friendly flash message.
 - Reloading a protected page after logout does not allow access.

Task #1 (2 pts.) – Login And Logout Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Show the login form code with the single username-or-email identifier field.
- Show the JavaScript `validate()` logic for login.
- Show the PHP login lookup, password verification, and session assignment logic.
- Show deployed qa browser evidence for login with email, login with username, invalid credentials, logout, and protected access after logout.
- For the login evidence, capturing the network request in developer tools is a good way to show the differences



Login form with one username-or-email identifier field and JavaScript validation logic.



PHP login lookup, password verification, role loading, and safe session assignment.

JavaScript validation logic.

safe session assignment.



```
const user = session.get('user');
if (user) {
  // Destroy the session and redirect to the login page
  session.destroy();
  res.redirect('/login');
} else {
  // Redirect to the login page
  res.redirect('/login');
}
```

Logout logic destroying the session and redirecting with a friendly confirmation message.



Successful login using an email address on the deployed QA site.



Successful logout showing session clearance and friendly confirmation on the deployed QA site.



Protected dashboard request redirecting a logged-out user to the QA login page.



Invalid login credentials rejected with user-friendly feedback on the deployed QA site.



Successful login using a username on the deployed QA site.

Part 2: URLs 100%

Details

- Paste the direct GitHub file link for `public_html/project/login.php`.
- Paste the direct GitHub file link for `public_html/project/logout.php`.
- Paste anticipated production URLs for the login and logout pages.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/qa/public_html/project/login.php 🍏

URL #2

https://github.com/micopher/mrc82-IT202-450-M2026/blob/qa/public_html/project/logout.php 🍏

URL #3

<https://mrc82-it202-450-m2026-prod.onrender.com/project/login.php> 🍏

URL #4

<https://mrc82-it202-450-m2026-prod.onrender.com/project/logout.php> 🍏

Part 3: Response 100%

Details

- Explain how the identifier can match either username or email.
- Explain how the password check works.
- Explain how logout clears access.

Response

Response

The login form uses one identifier field. PHP checks whether the value contains an @ symbol and validates it as an email or otherwise validates it as a username. The database query then searches for a matching username or email. The submitted password is checked against the stored password hash using `password_verify()`. After a successful login, safe user details and roles are stored in the session. Logout clears and destroys the session, creates a friendly confirmation message, and redirects the user to the login page, preventing continued access to protected pages.

Supports **bold**, *italic*, [links](#)(url), ``code``, etc.

572 chars

Section #5: (2 pts.) Profile

100%

- Goal: show that users can view and update profile information safely.
- Expected account-details behavior:
 - Profile view shows safe account information.
 - Username and email can be updated without the current password.
 - Duplicate username and email cases show specific, user-friendly feedback.
 - HTML, JavaScript, and PHP validation are all present.
- Expected password-change behavior:
 - Current password is required before saving a new password.
 - New password and confirm password are validated.
 - Successful password changes store a new hash.

Task #1 (2 pts.) – Profile Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Show the profile view and edit form code.
- Show the account-details update logic.
- Show the password-change logic.
- Show deployed qa browser evidence for profile view, account-details update, duplicate profile feedback, password-change failure with wrong current password, and successful password change.



Profile page showing the logged-in user's account details and password-change form on the deployed QA site.



Password-change form and JavaScript validation for current, new, and confirmed passwords.



Profile account-details form with username and email validation and account update logic.



PHP profile update logic with validation, prepared database update, session refresh, and duplicate handling.

Profile account-details form with username and email validation and escaped current values.

PHP profile update logic with validation, prepared database update, session refresh, and duplicate handling.



PHP password-change logic verifying the current password, hashing the new password, and updating the database.



Password change rejected because the current password was incorrect on the deployed QA site.



Successful password change confirmed on the deployed QA profile page.



Successful username and email update confirmed on the deployed QA profile page.



Duplicate username profile update rejected with specific feedback and sticky form values on the deployed QA site.



Duplicate email profile update rejected with specific feedback and sticky form values on the deployed QA site.

Part 2: URLs 100%

Details

- Paste the direct GitHub file link for `public_html/project/profile.php`.
- Paste the anticipated production URL for the profile page.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Milestone1-Part08-User-Login-Enhancement/public_html/project/profile.php 🍏

URL #2

<https://mrc82-it202-450-m2026-prod.onrender.com/project/profile.php> 🍏

Part 3: Response 100%

Details

- Explain the flow/process for changing username/email.
- Explain why password changes do require the current password.
- Explain how updated profile values persist after refresh.

Response

To change the username or email, the form validates both values before PHP runs a prepared database update. After a successful update, the new values are also saved in the user session, so the navigation and

To change the username or email, the form validates both values before PHP runs a prepared database update. After a successful update, the new values are also saved in the user session, so the navigation and profile page use the updated information. Password changes require the current password to prove that the logged-in user owns the account. PHP verifies it with `password_verify()`, validates the new passwords, and stores only a new bcrypt hash. Updated profile values remain after refresh because they are saved in the database and loaded again when the profile page opens.

Supports ****bold****, **italic**, [\[links\]\(url\)](#), ``code``, etc.

579 chars

Section #6: (1 pt.) Access Roles And Navigation

100%

- Goal: show that access rules are enforced by the server, reflected in navigation, and presented with your own project styling.
- Expected behavior:
 - Logged-out users cannot access protected pages.
 - Users without the required role cannot access role-protected pages.
 - Navigation changes based on login state.
 - The project uses customized CSS instead of the temporary lesson example styling.
 - Role pages from the lessons, such as create/list/assign roles, can satisfy the role-protected workflow.

Task #1 (0.67 pts.) – Access And Role Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Show the protected-page guard code.
- Show the role-protected guard code.
- Show the navigation code that changes based on login state.
- Show deployed **qa** browser evidence for a logged-out protected-page block, a wrong-role block, and a navigation state change.



Dashboard guard redirecting logged-out users to the login page before protected content loads.



Admin-only page calling the required-role guard before restricted content loads.



Non-admin user blocked from the role-protected page with friendly feedback on the deployed QA site.



Navigation code changing available links based on whether the user is logged in.





Role helper blocking logged-out users and users without the required role before restricted pages load.



Protected dashboard request redirecting a logged-out user to the QA login page.



Logged-in navigation showing Dashboard, Profile, and Logout links on the deployed QA site.

Part 2: URLs 100%

Details

- Paste the direct GitHub file link for `public_html/project/dashboard.php`.
- Paste the direct GitHub file link for `partials/nav.php`.
- Paste the direct GitHub file link for one role-protected admin page you tested.
- Paste anticipated production URLs for the protected and role-protected pages.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Milestone1-Part08-User-Login-Enhancement/public_html/project/dashboard.php 🍏

URL #2

<https://github.com/micopher/mrc82-IT202-450-M2026/blob/Milestone1-Part08-User-Login-Enhancement/partials/nav.php> 🍏

URL #3

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Milestone1-Part08-User-Login-Enhancement/public_html/project/admin.php 🍏

URL #4

<https://mrc82-it202-450-m2026-prod.onrender.com/project/dashboard.php> 🍏

URL #5

<https://mrc82-it202-450-m2026-prod.onrender.com/project/admin.php> 🍏

Part 3: Response 100%

Details

- Explain how the protected page checks login state.
- Explain how the role-protected page checks role access.
- Explain how navigation decides what to show.

Response

The dashboard checks `is_logged_in()` before loading protected content. If no user is logged in, PHP redirects the user to the login page and stops the script with `exit`. The admin page calls `require_role("Admin")`, which first confirms that the user is logged in and then checks the roles stored in the session. Users without the Admin role are redirected to the dashboard with a friendly permission message. The navigation always shows Home, then uses `is_logged_in()` to show Dashboard, Profile, and Logout for logged-in users or

Admin role are redirected to the dashboard with a friendly permission message. The navigation always shows Home, then uses `is_logged_in()` to show Dashboard, Profile, and Logout for logged-in users or Register and Login for logged-out users.

Supports **bold**, *italic*, [\[links\]\(url\)](#), ``code``, etc.

567 chars

🏠 Task #2 (0.33 pts.) – Project Styling Evidence

100%

📦 Combo #1

🖼️ Part 1: Images 100%

📋 Details

- Show the shared stylesheet or custom CSS changes.
- Show deployed **qa** browser evidence that the project no longer uses the temporary lesson example styling.



Custom project stylesheet defining the page background, navigation, links, headings, and centered form-card layout.



Deployed QA profile page showing the project's custom navigation, form cards, buttons, and flash-message styling.

🔗 Part 2: URLs 100%

📋 Details

- Paste the direct GitHub file link for `public_html/project/styles.css`.
- Paste the anticipated production URL for one styled project page.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Milestone1-Part08-User-Login-Enhancement/public_html/project/styles.css 🍏

URL #2

<https://mrc82-it202-450-m2026-prod.onrender.com/project/profile.php> 🍏

✍️ Part 3: Response 100%

📋 Details

- Explain what you changed in the CSS to make the project your own.

Response

I replaced the temporary lesson styling with a custom layout using a light-gray page background, a dark navigation bar, centered white form cards, rounded corners, shadows, and blue buttons with hover effects. I also added consistent spacing and styled success, danger, warning, and information flash messages with different colors so users can easily understand each result.

Supports **bold**, *italic*, [\[links\]\(url\)](#), ``code``, etc.

375 chars

Section #7: (1 pt.) Misc

100%

Task #1 (0.33 pts.) – GitHub Details

100%

Combo #1

Part 1: Images 100%

Details

- Screenshot the Commits tab of the pull request.
- The screenshot should show meaningful feature commits for Milestone 1.



Pull request Commits tab showing meaningful Milestone 1 account, login, profile, role, helper, and styling commits.

Part 2: URLs 100%

Details

- Paste the pull request URL.
 - The URL must end in `/pull/#`, where `#` is the pull request number; zero credit for this URL if it does not.

URL #1

<https://github.com/micopher/mrc82-IT202-450-M2026/pull/16> 🍏

Task #2 (0.33 pts.) – WakaTime Activity

100%

Details

- Visit the WakaTime dashboard.
- Open **Projects**.
- Find your repository.
- Screenshot the overall time near the repository name.
- Screenshot the file-level activity.
- The purpose is evidence of activity; the duration is not tied to the grade.
- The visual graphs are not required.





WakaTime project overview showing total development time and language activity for the repository.



WakaTime file-level activity showing time spent on Milestone 1 project files and branches.

Task #3 (0.33 pts.) – Reflection

100%

Details

- Answer each question with at least a few reasonable sentences.

Task #1 (0.11 pts.) – What did you learn during this assignment?

100%

Response

I learned how registration, login, sessions, roles, and profile updates work together. I also learned how PHP validation, database queries, and flash messages help make the site safer and easier to use.

Supports **bold**, *italic*, [links](url), `code`, etc.

202 chars

Task #2 (0.11 pts.) – What was the easiest part of the assignment?

100%

Response

The easiest part was testing the pages and taking screenshots once everything was working. It was pretty straightforward to check the forms, messages, and redirects.

Supports **bold**, *italic*, [links](url), `code`, etc.

165 chars

Task #3 (0.11 pts.) – What was the hardest part of the assignment?

100%

Response

The coding was the hardest part because it took A LOT of trial and error. Fixing validation, redirects, and login issues took time, but it helped me understand the code better, which is why I love this class, but am little sad because I wanted to take this class with a more generous time to complete.

Supports **bold**, *italic*, [links](url), `code`, etc.

301 chars

End of Assignment - submission has been recorded.

